

The DOM & Events Gauntlet

A Timed Vanilla JavaScript Lab

Duration: 90 minutes **Total:** 100 points **Mode:** Individual **Tools:** HTML/CSS/JS only

Rules of Engagement

- **No libraries, no frameworks.** No jQuery, no React, no Bootstrap JS. Pure DOM API and vanilla CSS/JS only.
- **No inline handlers.** `onclick="..."` in HTML is worth zero credit for that element. Use `addEventListener` exclusively.
- Elements that the task requires you to create **must be created and inserted using JavaScript** (`document.createElement`, `appendChild`, etc.) unless the task explicitly says to hand-author it in HTML.
- You may use the starter files provided (`index.html`, `styles.css`, `script.js`) or start from scratch, but the three-file structure must be preserved for submission.
- You are being timed. Partial credit is awarded generously for a genuine, testable attempt — a working `console.log` beats a silent crash.
- Test everything by actually clicking, typing, and dragging. “It should work” is not the same as “it works.”

This lab is intentionally tight on time. Read all five parts before writing code. Budget your 90 minutes using the suggested timings — they add up to 90, with zero slack. If you are stuck for more than 5 minutes on one sub-task, leave a `// TODO` comment describing your intended approach and move on; a documented plan earns partial credit, a blank space does not.

1 Part A — The Element Factory

(15 min) [15 pts]

Part A: Build it all with JavaScript

Given the JavaScript array below (place it at the top of `script.js`), dynamically generate a **product catalog** inside the empty `<div id="catalog"></div>` in `index.html`. Nothing in the catalog may be hand-written in the HTML file — it is all built with DOM methods.

```
const products = [
  { id: 1, name: "Mechanical Keyboard", price: 4999, inStock: true,
    tags: ["electronics", "peripherals"] },
  { id: 2, name: "Ceramic Mug", price: 349, inStock: false,
    tags: ["home", "kitchen"] },
  { id: 3, name: "Standing Desk", price: 12999, inStock: true,
    tags: ["furniture", "office"] },
];
```

For **every** product, create and append (in this exact nesting):

1. An `<article class="card">` wrapping the whole product.
2. An `<h3>` with the product name.
3. A `<p>` with the price, formatted as Rs. 4,999 (comma-separated, no decimals; you may use the rupee symbol instead of Rs. if you prefer).
4. A `<span>` badge reading In Stock or Out of Stock, with a CSS class `badge-in` or `badge-out` attached depending on `inStock`.
5. A `<ul>` of `<li>` elements, one per tag in `tags`.
6. A `<button data-id="...">` labelled Add to Cart that is **disabled** (real disabled attribute, not just styled) when `inStock` is false.

Additionally, above the cards, create a single `<table>` (with proper `<thead>/<tbody>`) summarizing all products: columns *Name*, *Price*, *Tags (comma-joined)*. Build every row and cell with `createElement`.

Constraint: you must use **event delegation** — attach exactly *one* click listener on `#catalog` (not one per button) that figures out which Add to Cart button was clicked via `event.target / closest()`, and logs the corresponding product's id to the console.

2 Part B — Mouse Events Arena

(20 min) [20 pts]

Part B: Every mouse event earns its keep

Work inside `<div id="mouse-arena"></div>`. Implement **all** of the following, each independently observable/testable:

1. **Live coordinate readout [3 pts]** — On `mousemove` inside the arena, update a `<p id="coords">` with the cursor's position *relative to the arena itself* (not the page). Use `getBoundingClientRect()`.
2. **Hover glow, no flicker [3 pts]** — A box (`#glow-box`) should get a class `.glowing` on `mouseenter` and lose it on `mouseleave`. It must *not* flicker when the cursor moves over child elements inside the box (i.e. do not use `mouseover/mouseout` naively).
3. **Click vs. double-click [3 pts]** — A counter div should increment on single `click`, but a `dblclick` should reset the counter to 0 *instead of* counting as two clicks. (Hint: you will need to suppress or account for the extra `click` events a double-click also fires.)
4. **Custom context menu [5 pts]** — On `contextmenu` inside the arena, `preventDefault()` the native browser menu and instead show your own small custom menu (a `<ul>` you create/toggle) with at least two fake options (e.g. "Delete", "Duplicate"), positioned at the click coordinates. Clicking anywhere else on the page must close it.
5. **Drag-to-move (manual, no HTML5 Drag API) [6 pts]** — Implement dragging of `#drag-box` using `mousedown + mousemove` (on document, not the box) + `mouseup`. While dragging, disable text selection (`user-select: none` toggled via JS or CSS) so the drag doesn't select page text.

3 Part C — Keyboard Events Arena

(20 min) [20 pts]

Part C: Keys, combos, and focus

Work inside `<div id="keyboard-arena"></div>`.

1. **Arrow-key navigation [6 pts]** — A small square `#player` must move by a fixed step (e.g. 10px) on `ArrowUp/Down/Left/Right` keydown, and must be **clamped** so it can never leave the bounds of its parent container. `preventDefault()` so the page itself doesn't scroll.
2. **Live character counter with a hard limit [4 pts]** — A `<textarea id="bio" maxlength="120">` must show "N / 120" below it, updating on every keystroke (`input`, *not* `keydown`, and explain in a code comment *why* `input` is the correct choice here). When N passes 100, the counter text must turn red via a toggled class.
3. **Key combo detection [5 pts]** — Detect `Ctrl+S` (or `Cmd+S` on Mac, i.e. check `e.ctrlKey` — `e.metaKey`—) anywhere on the page, `preventDefault()` the browser save dialog, and instead show a "Draft saved!" toast message that fades out after 2 seconds.
4. **Focus/blur styling [3 pts]** — Every `<input>` and the `<textarea>` in this section should get a class `.focused` added on `focus` and removed on `blur`, used to visibly highlight the active field.
5. **Konami-style sequence detector [2 pts]** — Track the last several keys pressed (an array, trimmed to a fixed length) and detect the sequence `["ArrowUp", "ArrowUp", "ArrowDown", "ArrowDown"]`. On success, log `"sequence unlocked"` and briefly flash the whole page background.

4 Part D — Transformation Lab

(20 min) [20 pts]

Part D: CSS transforms, driven by JS

Work inside `<div id="transform-lab"></div>`. All transforms must be applied via **CSS classes toggled by JS** (no directly setting `element.style.transform` strings in Parts 1–3 below — that’s only allowed in Part 4).

1. **Card flip [5 pts]** — A “card” with a front and back face flips 180° around the Y-axis (`rotateY`) when clicked, using `transform-style: preserve-3d` and `backface-visibility: hidden` on the faces, with a smooth `transition`. Clicking again flips it back.
2. **Hover scale + rotate combo [4 pts]** — On hover, an element should simultaneously `scale(1.1)` and `rotate(-3deg)` in a single `transform` property (not two separate ones), with `transform-origin` set to `center`, animated with a `transition`.
3. **Skewed slider track [4 pts]** — Build a small horizontal “progress” bar container that is itself `skewX(-15deg)`, but the text/number label inside it must be counter-skewed (`skewX(15deg)`) so it reads upright. Use a `<button>` to bump the fill percentage (and its width) up by 10% each click, capped at 100%.
4. **JS-driven translate with bounds checking [7 pts]** — A box moves along a track when a slider (`<input type="range">`) is dragged; here you *do* set `element.style.transform = 'translateX(...)'` directly from JS on the slider’s input event, computing the pixel offset from the slider’s percentage and the track’s width via `getBoundingClientRect()`. Must work correctly if the browser window is resized (recompute track width, don’t hardcode it).

Reminder: combining transforms in one declaration, e.g. `transform: scale(1.1)rotate(-3deg);`, applies them in the order written — order matters and is worth checking if your result looks “off”.

5 Part E — Integration Boss Fight

(15 min) [25 pts]

Part E: Everything, at once

Work inside `<div id="boss-fight"></div>`. Build a tiny “**drag-and-drop task board**” with exactly two columns, **To Do** and **Done**, that satisfies *all* of the following simultaneously:

1. [6 pts] At least 4 task “cards” are generated from a JS array of task-name strings (element creation, as in Part A) and appended into the **To Do** column.
2. [7 pts] Each card can be **dragged with the mouse** (manual `mousedown/mousemove/mouseup`, as in Part B) from **To Do** into **Done**. Dropping is column-aware: if released while overlapping the **Done** column’s bounding box, the card is re-parented into that column; otherwise it snaps back.
3. [6 pts] Whichever card is currently focused/selected can also be moved to the other column using the **keyboard** (e.g. pressing `Enter` or `M` toggles its column) — keyboard and mouse must both be fully functional, independently.
4. [4 pts] Every card `scale()`s up slightly on `mouseenter` and plays a brief “pop” transform-based animation (`transform + transition`, or a `@keyframes` animation) whenever it successfully changes columns, by either method.
5. [2 pts] A live counter (“Done: N / total”) updates automatically whenever a card moves, however it was moved.

This part deliberately forces you to reuse and connect code from Parts A–D. If you are running low on time, a version that supports *only* keyboard-based column switching (skipping true drag physics) is still worth significant partial credit — do not leave this part completely blank.

Submission Checklist

- ☐ `index.html`, `styles.css`, `script.js` — three separate files, properly linked.
- ☐ No console errors on page load or during normal interaction.
- ☐ No inline `style=` or `on*=` attributes used to satisfy a requirement.
- ☐ Every unfinished sub-task has a `// TODO` explaining your intended approach.

Total: 100 points across 90 minutes. Good luck — and read before you type.